Operating Systems
Week 9 Lecture 9

Junaid Abdul Wahid

Previous

- Resource Allocation Graph
- Banker’s Algorithm

Today Discussion

- Resource Allocation Graph
- Banker Algorithm
- Safety Algorithm
-

Banker Algorithm

The Banker’s Algorithm is used in operating systems to avoid deadlocks by ensuring that a system remains in a safe state before allocating resources to processes (or threads). It simulates resource allocation for each possible request, and only allocates if the system stays safe.

Data Structures for the Banker's Algorithm

Let n = number of processes, and m = number of resources types.

Available: Vector of length m. If available $[j] = k$, there are k instances of resource type R_j available

Max: $n \times m$ matrix. If $Max[i,j] = k$, then process T_i may request at most k instances of resource type R_j

Allocation: $n \times m$ matrix. If $Allocation[i,j] = k$ then T_i is currently allocated k instances of R_j

Need: $n \times m$ matrix. If $Need[i,j] = k$, then T_i may need k more instances of R_j to complete its task

$$Need[i,j] = Max[i,j] - Allocation[i,j]$$

Data Structures for the Banker's Algorithm

For each thread (process), we define:
Available: Vector of available resources.
Max: Maximum demand of each process.
Allocation: Resources currently allocated to a process.
Need: Remaining resources needed = Max - Allocation.

A system is in a **safe state** if there exists a sequence of processes such that each process can finish using available + allocated resources.

Safety Algorithm

1. Let ***Work*** and ***Finish*** be vectors of length *m* and *n*, respectively. Initialize:

   Work* = *Available

   ***Finish* [*i*] = *false* for *i* = 0, 1, …, *n*- 1**

2. Find an ***i*** such that both:

   (a) ***Finish* [*i*] = *false***

   (b) $Need_i \leq Work$

   If no such ***i*** exists, go to step 4

3. $Work = Work + Allocation_i$

   $Finish[i] = true$

   go to step 2

4. If ***Finish* [*i*] == *true*** for all ***i***, then the system is in a safe state

Resource-Request for Process P_i

Request$_i$ = request vector for process T_i. If ***Request***$_i$ **[*j*]** = ***k*** then process T_i wants ***k*** instances of resource type R_j

1. If ***Request***$_i \leq$ ***Need***$_i$ go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim
2. If ***Request***$_i \leq$ ***Available***, go to step 3. Otherwise T_i must wait, since resources are not available
3. Pretend to allocate requested resources to T_i by modifying the state as follows:

Available = Available – Request$_i$***;***
Allocation$_i$ ***= Allocation***$_i$ ***+ Request***$_i$***;***
Need$_i$ ***= Need***$_i$ ***– Request***$_i$***;***

- If safe $\Rightarrow$ the resources are allocated to T_i
- If unsafe $\Rightarrow$ T_i must wait, and the old resource-allocation state is restored

Example of Banker's Algorithm

5 threads T_0 through T_4;

3 resource types:

A (10 instances), *B* (5instances), and *C* (7 instances)

Snapshot at time T_0:

	Allocation	*Max*	*Available*
	A B C	*A B C*	*A B C*
T_0	0 1 0	7 5 3	3 3 2
T_1	2 0 0	3 2 2	
T_2	3 0 2	9 0 2	
T_3	2 1 1	2 2 2	
T_4	0 0 2	4 3 3	

Example (Cont.)

The content of the matrix ***Need*** is defined to be ***Max - Allocation***

	Need		
	A	*B*	*C*
T_0	7	4	3
T_1	1	2	2
T_2	6	0	0
T_3	0	1	1
T_4	4	3	1

The system is in a safe state since the sequence < T_1, T_3, T_4, T_0, T_2> satisfies safety criteria

Example (Cont.)

We use **Available = [3, 3, 2]** and try to find a **safe sequence**.

Step 1: Find a process whose **Need ≤ Available**
•T1: Need = [1, 2, 2] ☑ → Run T1 → Add its allocation [2, 0, 0] to Available → New Available = [5, 3, 2]

Step 2:
•T3: Need = [0, 1, 1] ☑ → Run T3 → Add its allocation [2, 1, 1] → Available = [7, 4, 3]

Step 3:
•T4: Need = [4, 3, 1] ☑ → Run T4 → Add its allocation [0, 0, 2] → Available = [7, 4, 5]

Step 4:
•T0: Need = [7, 4, 3] ☑ → Run T0 → Add its allocation [0, 1, 0] → Available = [7, 5, 5]

Step 5:
•T2: Need = [6, 0, 0] ☑ → Run T2 → Add its allocation [3, 0, 2] → Available = [10, 5, 7]
🎉 **All processes finished. Safe sequence = [T1, T3, T4, T0, T2]**

Example: P_1 Request (1,0,2)

Check that Request ≤ Available (that is, (1,0,2) ≤ (3,3,2) ⇒ true

	Allocation	*Need*	*Available*
	A B C	*A B C*	*A B C*
T_0	0 1 0	7 4 3	2 3 0
T_1	3 0 2	0 2 0	
T_2	3 0 2	6 0 0	
T_3	2 1 1	0 1 1	
T_4	0 0 2	4 3 1	

Executing safety algorithm shows that sequence < T_1, T_3, T_4, T_0, T_2> satisfies safety requirement

Can request for (3,3,0) by T_4 be granted?

Can request for (0,2,0) by T_0 be granted?

Example: P_1 Request (1,0,2)

- **T1**: Need = [0, 2, 0] ☑ → Run T1 → Release [3, 0, 2] → Work = [5, 3, 2]
- **T3**: Need = [0, 1, 1] ☑ → Run T3 → Release [2, 1, 1] → Work = [7, 4, 3]
- **T4**: Need = [4, 3, 1] ☑ → Run T4 → Release [0, 0, 2] → Work = [7, 4, 5]
- **T0**: Need = [7, 4, 3] ☑ → Run T0 → Release [0, 1, 0] → Work = [7, 5, 5]
- **T2**: Need = [6, 0, 0] ☑ → Run T2 → Release [3, 0, 2] → Work = [10, 5, 7]

- Safe Sequence = [T1, T3, T4, T0, T2]